

Retail E-Commerce Sales Analytics Pipeline

Databricks • Delta Lake • Medallion Architecture • SCD Type 2

CEI Intern Assignment Guide

Table of Contents

1. Problem Statement

You are building a production-style retail analytics pipeline for an e-commerce company. The company has historical batch CSV data and receives daily incremental files. The data is dirty: null customer/product IDs, duplicate orders, invalid dates, text inside numeric columns, updated customer/product attributes, late-arriving orders, and a schema change in day 3 incremental orders. The pipeline must ingest raw files, clean and standardize them, maintain slowly changing customer and product dimensions using SCD Type 2, and produce analytics-ready Gold tables for revenue, order, customer, product, and store metrics.

2. Dataset Package

Batch Files

Located at: `datasets/batch/`

File	Purpose	Notes
<code>orders_batch.csv</code>	Historical fact orders	12,000+ base rows plus duplicate and corrupted records
<code>customers_batch.csv</code>	Historical customer dimension	Includes nulls, duplicates, invalid signup dates
<code>products_batch.csv</code>	Historical product dimension	Includes unit_price anomalies like 'unknown'
<code>stores_batch.csv</code>	Store dimension	Includes duplicate records

Incremental Files

Located at: `datasets/incremental/day_YYYY-MM-DD/` Each day contains:

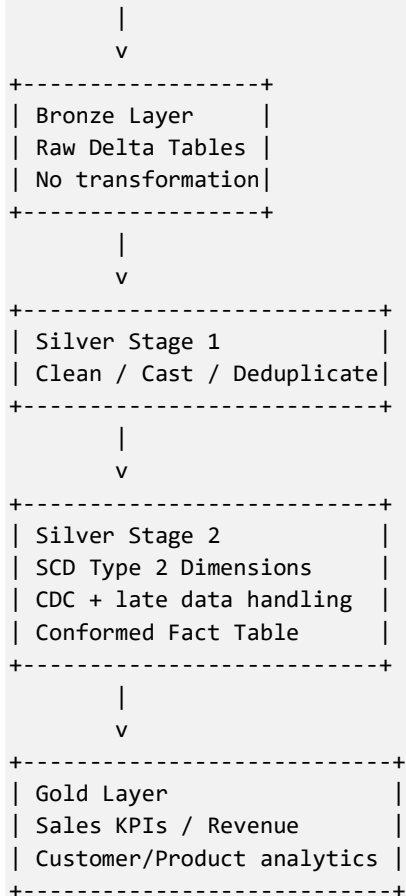
File Pattern	Purpose
<code>orders_incremental_YYYY-MM-DD.csv</code>	Around 2,000 daily orders with late arriving, duplicate, and

	corrupted rows
customers_cdc_YYYY-MM-DD.csv	Customer inserts/updates for SCD Type 2
products_cdc_YYYY-MM-DD.csv	Product updates for SCD Type 2

Note: Day 2026-04-26 includes a schema change in orders: new column `coupon_code`.

3. Architecture Diagram

CSV Files in Unity Catalog Volume / DBFS



4. Recommended Databricks Setup

Workspace and Cluster

Use a Databricks workspace with Unity Catalog enabled if available.

Recommended cluster configuration for this project:

```

Databricks Runtime: 14.3 LTS or above
Worker type: small general-purpose node is enough
Workers: 1-2
Autoscaling: optional
Photon: enabled if available
Unity Catalog: enabled if your workspace supports it
  
```

Suggested Catalog / Schema / Volume Layout

```
CREATE CATALOG IF NOT EXISTS retail_demo;  
CREATE SCHEMA IF NOT EXISTS retail_demo.raw;  
CREATE SCHEMA IF NOT EXISTS retail_demo.silver;  
CREATE SCHEMA IF NOT EXISTS retail_demo.gold;  
  
CREATE VOLUME IF NOT EXISTS retail_demo.raw.retail_files;
```

Upload the project folder into a Unity Catalog volume or DBFS.

5. Path Configuration

Set up the following variables to use across your notebooks:

```
CATALOG      = "retail_demo"  
RAW_SCHEMA   = "raw"  
SILVER_SCHEMA = "silver"  
GOLD_SCHEMA  = "gold"  
  
BASE_PATH    = "/Volumes/retail_demo/raw/retail_files/retail_delta_project"  
BATCH_PATH   = f"{BASE_PATH}/datasets/batch"  
INCR_PATH    = f"{BASE_PATH}/datasets/incremental"  
CHECKPOINT_PATH = f"{BASE_PATH}/_checkpoints"  
SCHEMA_PATH  = f"{BASE_PATH}/_schemas"
```

6. Bronze Layer — Raw Ingestion

Why Bronze?

Bronze keeps the raw source exactly as received. This acts as a historical archive, providing auditability, replay capability, and a safe landing zone before applying business rules. If transformation logic changes later, you can always reprocess from Bronze.

Tasks

- Ingest the historical batch CSV files for orders, customers, products, and stores into Delta tables in the raw schema.
- Ingest the incremental/CDC files (simulating a daily feed or streaming source) into their respective Bronze tables.
- Add audit metadata columns to every table: `source_file` (name of the file read from), `ingestion_ts` (timestamp of when the record was ingested), and `load_type` (e.g., "batch" or "incremental").

Hints

- **Data Types:** Do not try to enforce strict schemas or parse dates here. Read everything flexibly (e.g., as strings) to avoid job failures on malformed rows.
- **Incremental Loads:** Think about Databricks features that help automatically process only new files in a directory without tracking state manually.

- **Metadata:** Look into built-in functions that can grab the current timestamp and the name of the input file dynamically during the read operation.

7. Silver Stage 1 — Cleaning, Casting, Deduplication, Quarantine

Why Silver Stage 1?

This layer is where data quality is enforced. It converts raw, untyped data into structured, typed columns, removes duplicates, identifies invalid records, and separates them into quarantine tables for further investigation, ensuring only trusted data moves forward.

Tasks

- **Orders:** Convert timestamps, extract numeric values from strings (e.g., removing currency symbols or text from `unit_price`), identify bad records (null IDs or essential fields), and deduplicate.
- **Customers & Products:** Parse dates, handle null categorical fields (e.g., set to "Unknown"), identify bad records, and deduplicate.
- **Stores:** Deduplicate records based on the store identifier.

Hints

- **Cleaning:** Use regular expressions to replace or extract specific patterns (like keeping only digits and decimals in a price field) before casting the data type.
- **Deduplication:** A simple `distinct` operation might not be enough if a record was updated over time. Consider partitioning your data by the unique identifier, ordering by your ingestion timestamp (newest first), and picking the top record for that identifier.

8. Silver Stage 2 — SCD Type 2 Dimensions

Why SCD Type 2?

Slowly Changing Dimension (SCD) Type 2 keeps historical changes of attributes. If a customer moves to a new city or changes their segment, analytics must be able to answer: "What was the customer's city at the exact time they placed the order?" SCD2 solves this by adding effective date ranges to each record.

Tasks

- Create Customer and Product dimensions tracking historical changes.
- Add SCD Type 2 tracking columns: `surrogate_key` (system-generated unique integer key), `effective_start_date`, `effective_end_date`, `is_current`, and a `hash_value` representing the state of the attributes.
- Implement the logic to merge incoming CDC data: expire the currently active record if attributes have changed, and insert the new updated record.

Hints

- **Detecting Change:** Instead of comparing every single column to see if an update occurred, concatenate the columns you care about and generate a hash. Compare the incoming hash with the active record's hash.

- **Dates:** The `effective_start_date` is usually the date the change occurred (from the source). The `effective_end_date` for an active record should be a far-future date (like 9999-12-31).
- **Late Arriving Data:** If late data arrives, simply add and process it. It is not an issue; the architecture is designed to accommodate it.
- **Surrogate Keys:** Generate a surrogate key (e.g., using `monotonically_increasing_id()` or a hash-based integer) for each dimension row. This is a system-assigned, meaningless integer that uniquely identifies each physical row in the dimension table — including historical versions of the same business entity. The fact table references this surrogate key, not the natural business key, to correctly resolve the exact version of the dimension at the time of the transaction.

Confusing Topic: The SCD2 MERGE Pattern

Implementing SCD Type 2 with a MERGE operation can be tricky because a single update from the source actually requires two actions in the target:

- Update the existing active row to close its time window (`is_current = false`, set `effective_end_date`).
- Insert a new row with the new values and a new open time window.

Because standard MERGE only allows one action per matched row, a common pattern is to join the incoming data with the target table to isolate the changed records, and then perform a two-step operation (or use specific Delta features if available):

```
-- Conceptual two-step MERGE approach
-- Step 1: Expire old rows
MERGE INTO target_table t
USING changed_records s
ON t.id = s.id AND t.is_current = true
WHEN MATCHED AND t.hash <> s.hash THEN
    UPDATE SET t.effective_end_date = [new_start_date - 1], t.is_current = false;

-- Step 2: Insert new rows
MERGE INTO target_table t
USING changed_records s
ON t.id = s.id AND t.effective_start_date = s.effective_start_date
WHEN NOT MATCHED THEN
    INSERT *;
```

9. Final Fact Table

Why a Conformed Fact Table?

The fact table brings together the transactional events (orders) and the descriptive context (dimensions) into a single, unified view. Joining them correctly ensures that the metrics are accurate according to the historical state of the business.

Tasks

- Join the cleaned, watermarked orders with the Customer, Product, and Store dimensions.
- Select the necessary keys, attributes, and measures required for downstream reporting.

- Use the surrogate keys from the dimension tables (not the natural business keys) as foreign keys in the fact table. This ensures the fact table correctly references the exact historical snapshot of each dimension at the time the order was placed.

Hints

- **Crucial Join Logic:** When joining facts to SCD Type 2 dimensions, do not join on the natural business key alone. Instead, look up the correct dimension row using the business key AND a point-in-time date range check (transaction date must fall between `effective_start_date` and `effective_end_date`), then store the resolved surrogate key as the foreign key in the fact table. This is the standard star schema pattern for correct historical reporting.
- **Surrogate Key as Foreign Key:** The fact table should contain one surrogate key column per dimension (e.g., `customer_sk`, `product_sk`, `store_sk`). These are resolved at load time by performing the point-in-time SCD2 join, and they make downstream Gold aggregations significantly faster and simpler.

10. Gold Layer — Analytics Tables

Why Gold?

The Gold layer represents the final presentation layer. It contains highly refined, aggregated data built specifically to power dashboards, reports, and high-level business queries quickly and efficiently.

Tasks

- Daily Sales: Calculate total orders, total revenue, total units, and average order value per day.
- Category Sales: Calculate orders, revenue, and units sold per product category, ordered by revenue.
- Segment Sales: Calculate unique customers, orders, and revenue per customer segment.
- Region Sales: Calculate orders, revenue, and units per store region.

Hints

- Use standard grouping and aggregation functions. Think about the difference between counting rows and counting unique distinct values when calculating things like "unique customers".

11. Delta Lake Features to Demonstrate

Ensure you know how to leverage these Delta Lake features once your pipeline is built:

- **Table History:** How to view the transaction log of your tables.
- **Time Travel:** How to query a table as it existed at a specific point in time or version.
- **Schema Evolution:** Day 3 data introduces a new column (`coupon_code`). Configure your writes to automatically accept and evolve the table schema without failing.

12. Common Errors to Watch Out For

Error	Reason
PATH_NOT_FOUND	Uploaded folder path is wrong. Check your Volume or DBFS paths.
Failed to merge incompatible data types	You are trying to insert a string into an integer column. Fix casting in the Silver layer.
Detected schema change	A new column arrived (like coupon_code). You need to enable schema evolution.
Duplicate rows after incremental run	You missed the deduplication step or your merge keys are incorrect.
SCD old row not closed	Your hash comparison is failing, or you are updating the wrong row.

13. Best Practices Checklist

1. Keep Bronze raw and immutable.
2. Read dirty CSV columns flexibly in Bronze (e.g. as strings).
3. Apply schema casting and validation only in Silver.
4. Use business keys for deduplication (order_id, customer_id, product_id).
5. Use cryptographic hashes for SCD Type 2 change detection.
6. Join facts to SCD dimensions using point-in-time date ranges.
7. Use surrogate keys (not natural business keys) as foreign keys in the fact table.
8. Maintain isolated checkpoint paths for streaming/incremental workloads.
9. Keep Gold tables small, aggregated, and dashboard-ready.

14. Expected Final Deliverables

By the end of this project, you should have populated the following tables:

```
retail_demo.raw.bronze_orders
retail_demo.raw.bronze_orders_incremental
retail_demo.raw.bronze_customers
retail_demo.raw.bronze_customers_cdc
retail_demo.raw.bronze_products
retail_demo.raw.bronze_products_cdc
retail_demo.raw.bronze_stores

retail_demo.silver.silver1_orders_clean
retail_demo.silver.silver1_customers_clean
retail_demo.silver.silver1_products_clean
retail_demo.silver.dim_customer_scd2
retail_demo.silver.dim_product_scd2
retail_demo.silver.dim_store

retail_demo.gold.gold_daily_sales
retail_demo.gold.gold_category_sales
```

```
retail_demo.gold.gold_segment_sales  
retail_demo.gold.gold_region_sales  
retail_demo.gold.fact_orders
```